

KubeKG-Forecast: A Dynamic Knowledge Graph and Deep Graph Neural Network Framework for Failure-Rate Prediction in Kubernetes-Based Big-Data Platforms

Original SCI-Style Research Manuscript Draft

Integrating the core solutions of GATAKU and dynamic heterogeneous graph-based Kubernetes resilience

Prepared July 2026

Abstract

Modern big-data platforms increasingly depend on containerized and distributed computing infrastructures such as Kubernetes to orchestrate microservices, data-processing pipelines, storage services, and artificial-intelligence workloads. Although Kubernetes provides self-healing and reactive resource-management mechanisms, these mechanisms generally respond only after a failure or abnormal state has already occurred. Existing anomaly-detection approaches also tend to analyze individual metrics independently, ignore the heterogeneous relationships between platform components, or focus on current anomaly classification rather than explicit prediction of future failure probability. This paper proposes KubeKG-Forecast, a unified framework for anomaly detection, root-cause localization, failure-propagation analysis, and multi-horizon failure-rate prediction in Kubernetes-based big-data platforms. The framework synthesizes two complementary research directions: graph neural network-based anomaly detection from inter-container communication data and dynamic heterogeneous graph-based representation of Kubernetes resources with unsupervised and supervised anomaly learning. Kubernetes Nodes, Pods, Containers, Deployments, ReplicaSets, Services, Namespaces, storage resources, and communication flows are represented as a continuously evolving temporal knowledge graph. Prometheus and Thanos provide infrastructure telemetry, while Cilium and Hubble provide network-flow information. Neo4j maintains the graph representation, and an Isolation Forest-based weak-supervision stage produces high-confidence anomaly labels when expert annotations are unavailable. A heterogeneous relational graph attention network learns spatial dependencies among platform components, while a temporal recurrent or attention-based encoder captures degradation trajectories. The framework predicts current anomaly status, component-specific future failure probability, discrete failure hazard, and system-level failure rate over multiple forecasting horizons. An explainability layer based on graph attention, GNNExplainer, and knowledge-graph traversal identifies influential metrics, neighboring components, and possible failure-propagation paths. The resulting architecture extends reactive Kubernetes monitoring toward proactive, explainable, topology-aware reliability management for large-scale cloud and big-data systems.

Keywords: Kubernetes; knowledge graph; graph neural network; graph attention network; failure-rate prediction; anomaly detection; predictive maintenance; microservices; AIOps; Neo4j; Prometheus; Cilium; root-cause analysis.

1. Introduction

1.1 Background and Motivation

The rapid growth of cloud computing, artificial intelligence, real-time analytics, and large-scale data processing has created increasingly complex distributed computing infrastructures. Modern big-data platforms frequently consist of hundreds or thousands of services, containers, storage volumes, virtual machines, network connections, and computing nodes. Frameworks such as Kubernetes have become central to the deployment and orchestration of these systems because they provide automatic scheduling, service discovery, horizontal scaling, rolling updates, load balancing, and self-healing.

However, the same flexibility that makes Kubernetes attractive also introduces significant operational complexity. Pods are continuously created and removed. Containers may be restarted automatically. Services communicate through changing network paths. Applications may be distributed across multiple physical or virtual nodes. Resource pressure on one component may propagate to several dependent services. Consequently, the overall platform is dynamic, heterogeneous, highly interconnected, and difficult to monitor using conventional flat tabular models.

Kubernetes includes several native resilience mechanisms. For example, failed containers can be restarted, unhealthy Pods can be replaced, Deployments can recreate unavailable replicas, and the Horizontal Pod Autoscaler can increase or decrease replica counts in response to selected metrics. Nevertheless, most of these mechanisms are primarily reactive. They take action after an undesirable condition has been detected rather than estimating the probability that a component will fail in the near future.

The limitation is important in large-scale big-data platforms. A Hadoop DataNode with increasing disk utilization, a Kubernetes Node experiencing growing memory pressure, a Pod exhibiting escalating restart frequency, or a service showing abnormal dependency behavior may continue operating for a period before an actual failure occurs. Detecting the early degradation trajectory and estimating the future failure probability would allow preventive migration, rescheduling, scaling, repair, or operator intervention.

Traditional machine-learning approaches commonly analyze each component using independent metric vectors. However, a Kubernetes platform is fundamentally relational. A Pod executes on a Node, belongs to a Namespace, is managed by a ReplicaSet, is controlled by a Deployment, communicates with Services, accesses volumes, and sends network traffic to other Pods. Ignoring such relationships removes important contextual information.

Graph-based methods provide a natural solution. A Kubernetes cluster can be represented as a dynamic heterogeneous graph in which vertices correspond to platform resources and edges correspond to operational, structural, dependency, or communication relationships. Graph neural networks can then propagate information between related components and learn patterns that conventional models cannot easily capture.

Two recent research directions provide particularly important foundations. The first, represented by the GATAKU approach, applies a Graph Convolutional Network to Kubernetes network traffic and demonstrates that graph neural networks can accurately identify anomalous communication patterns. The second represents Kubernetes resources as a dynamic heterogeneous graph stored in Neo4j and combines Prometheus monitoring with unsupervised and supervised machine learning for anomaly detection, anomaly prediction, and hierarchical localization of defective Pods and Nodes.

Although these approaches demonstrate strong potential, several research gaps remain. GATAKU constructs an artificial k-nearest-neighbor graph from network-flow observations instead of using the native semantic relationships of Kubernetes resources. Its primary objective is present-time cybersecurity anomaly classification rather than operational failure forecasting. Conversely, the dynamic graph-based resilience approach models Kubernetes topology and operational metrics but uses conventional machine-learning models rather than graph neural networks capable of learning directly from graph structure. Furthermore, neither solution explicitly estimates component-specific failure rates over multiple future time horizons.

This paper proposes KubeKG-Forecast, an integrated architecture that addresses these limitations.

1.2 Research Objective

The main objective of this research is to develop a method that can continuously answer four operational questions:

1. Is a Kubernetes component currently behaving abnormally?
2. Which component or dependency is most likely responsible for the abnormal state?
3. How is the abnormal condition likely to propagate through the platform graph?
4. What is the probability and estimated rate of component failure within future horizons such as 5 minutes, 30 minutes, 1 hour, or 6 hours?

The proposed framework combines dynamic knowledge graphs, infrastructure telemetry, network-flow monitoring, weakly supervised anomaly labeling, heterogeneous graph neural networks, temporal deep learning, and explainable graph reasoning.

1.3 Main Contributions

The principal contributions of this work are as follows:

5. A unified dynamic temporal knowledge graph is proposed for representing Kubernetes resources, native orchestration relationships, service dependencies, infrastructure metrics, and real-time communication flows.
6. The approach integrates Prometheus/Thanos resource telemetry with Cilium/Hubble network observability, thereby combining the infrastructure-monitoring strengths of the dynamic heterogeneous graph approach with the network-flow analysis strengths of GATAKU.
7. A weakly supervised two-stage learning strategy is introduced. An unsupervised Isolation Forest initially identifies high-confidence abnormal and normal observations. These pseudo-labels, together with any available verified incident labels, supervise a deep heterogeneous graph neural network.
8. A heterogeneous relational graph attention network replaces both the artificially generated k-nearest-neighbor graph used in the network-anomaly solution and the independent conventional classifiers used in the dynamic graph solution.
9. A temporal forecasting layer estimates current anomaly probability, future component-failure probability, discrete failure hazard, and failure rate over multiple prediction horizons.
10. The proposed system performs graph-based root-cause localization and failure-propagation reasoning by tracing influential neighboring components and dependency paths.
11. An explainability layer integrates graph attention, GNNExplainer, metric-level feature attribution, and Neo4j graph visualization to provide human-understandable explanations for predictions.

2. Related Work

2.1 Kubernetes Anomaly Detection

Kubernetes platforms expose large amounts of telemetry describing CPU utilization, memory consumption, network activity, container restarts, API-server behavior, storage utilization, and application performance. Traditional Kubernetes

anomaly-detection techniques have applied statistical thresholds, clustering, Isolation Forests, One-Class Support Vector Machines, support-vector machines, decision trees, and neural networks.

Such techniques are useful but frequently assume that observations are independent. A metric vector representing Pod p_i , for example, may be classified without considering the condition of the Node on which it executes, the Deployment controlling it, or the services with which it communicates. This independence assumption does not accurately reflect distributed cloud systems.

Furthermore, many anomaly-detection systems identify only whether an observation is unusual at the present time. For proactive reliability management, the more important objective is often to determine whether apparently small deviations are signs of future degradation.

2.2 GATAKU: Graph Neural Network-Based Network Anomaly Detection

Osswald et al. proposed GATAKU, a Graph Neural Network-based Anomaly Detection method for Kubernetes environments. The system deploys Kubernetes on an OpenStack infrastructure and uses Cilium and OpenTelemetry-compatible monitoring to collect inter-container network data.

The study simulates three cybersecurity scenarios: denial-of-service attacks, TCP port scans, and OWASP ZAP vulnerability scanning. Collected network flows are processed through IP encoding, string encoding, numerical normalization, Boolean encoding, and an engineered diversity-index feature measuring the number of unique destination ports accessed by a source within a sliding temporal window.

Because individual network records do not inherently form a graph, GATAKU applies k-nearest neighbors to connect observations with similar features. A three-layer Graph Convolutional Network then performs node classification. The reported GCN architecture contains three graph convolution layers, ReLU activation, dropout regularization, and a Softmax classification output. The study also applies GNNExplainer to determine influential nodes, edges, and input features.

GATAKU demonstrates the effectiveness of graph neural networks in Kubernetes anomaly detection. Its main limitations in the context of failure-rate forecasting are that the graph is artificially generated from feature similarity, Kubernetes semantic relationships are not fully represented, and the output primarily indicates present cybersecurity anomaly classes.

2.3 Dynamic Heterogeneous Graph-Based Kubernetes Resilience

Anemogiannis et al. proposed a Kubernetes anomaly-detection and prediction architecture in which the cluster is represented as a dynamic heterogeneous graph. Kubernetes components such as Nodes, Namespaces, Deployments, ReplicaSets, Pods, Containers, Services, StatefulSets, Ports, and Labels are stored in a Neo4j database.

Prometheus collects resource telemetry, including CPU saturation and memory utilization. Thanos supports long-term storage, global querying, and distributed metric aggregation.

The approach divides anomaly learning into two stages. During the first stage, unsupervised models identify historical observations that deviate from expected behavior. Isolation Forest, DBSCAN, and One-Class SVM are compared. During the second stage, pseudo-labeled observations are used to train supervised models capable of predicting the anomaly status of future observations. Logistic Regression, SVM, and Decision Trees are evaluated.

The graph is processed hierarchically because some Kubernetes resources, such as Deployments and ReplicaSets, act as aggregation layers rather than directly monitored physical resources. By evaluating lower-level resources first, the method can identify a problematic Pod and then trace its associated ReplicaSet, Deployment, Namespace, and hosting Node.

The approach successfully demonstrates topology-aware anomaly localization. Nevertheless, because prediction models are independently applied to resource-type datasets, they do not directly learn information through graph message passing. Temporal degradation trajectories and explicit failure-rate forecasting also remain future research opportunities.

2.4 Research Gap

The literature reveals four principal gaps.

12. Existing approaches typically emphasize either graph neural network analysis of network records or semantic graph representation of Kubernetes topology, but rarely integrate both.
13. An artificial similarity graph does not fully express Kubernetes semantics. Two network packets may be similar in feature space without belonging to operationally related services. Conversely, two topologically dependent Pods may exhibit different raw metrics while remaining strongly connected from a failure-propagation perspective.

14. Current anomaly detection does not necessarily equal future failure prediction. A component may temporarily exhibit a statistical anomaly without failing, while slow degradation may lead to failure without generating an extreme instantaneous outlier.
15. Most methods provide binary or multiclass anomaly labels rather than a calibrated quantity such as the conditional probability that a component fails within a future horizon given recent graph history.

$$P(T_v \leq t + \Delta \mid \mathcal{G}_{(t-L:t)})$$

The proposed KubeKG-Forecast framework is designed to address these gaps.

3. Problem Formulation

3.1 Dynamic Heterogeneous Platform Graph

At observation time t , the Kubernetes environment is represented by:

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{X}_t, \mathcal{R})$$

where $\mathcal{V}_t$ is the set of resource vertices, $\mathcal{E}_t$ is the set of directed or undirected edges, $\mathcal{X}_t$ contains node and edge attributes, and $\mathcal{R}$ is the set of relation types.

$$\tau(v) \in \{\text{Cluster, Node, Namespace, Deployment, ReplicaSet, Pod, Container, Service, Volume}\}$$

Possible relationships include runsOn, contains, manages, owns, exposes, communicatesWith, mounts, and dependsOn. Example triples include:

$$\begin{aligned} &(\text{Pod}_{17}, \text{runsOn}, \text{Node}_3) \\ &(\text{ReplicaSet}_4, \text{manages}, \text{Pod}_{17}) \\ &(\text{Service}_A, \text{communicatesWith}, \text{Service}_B) \\ &(\text{Pod}_{17}, \text{mounts}, \text{Volume}_8) \end{aligned}$$

Because Kubernetes resources appear, disappear, restart, migrate, and change relationships, $\mathcal{G}_t$ is dynamic.

3.2 Node Feature Representation

$$x_v^t = [x_{\text{resource}}, x_{\text{network}}, x_{\text{event}}, x_{\text{topology}}]$$

Resource metrics may include CPU utilization, CPU throttling, memory working set, memory saturation, disk utilization, disk I/O latency, filesystem capacity, network throughput, restart count, request rate, request latency, and error rate.

Network features may include source and destination endpoints, source and destination ports, protocol, connection count, packet count, transmitted bytes, dropped packets, retransmissions, flow duration, and diversity index. The diversity index inspired by GATAKU can be expressed as:

$$DI_i(t) = |\{d_p : t - w \leq t_j \leq t, s_j = i\}|$$

where w is a sliding time window and d_p represents unique destination ports contacted by source i .

Event features may contain OOMKilled occurrences, CrashLoopBackOff events, FailedScheduling events, eviction events, image-pull errors, NodeNotReady events, readiness-probe failures, and liveness-probe failures. Topology features can include node degree, number of downstream dependencies, number of upstream dependencies, communication centrality, deployment replica ratio, and number of affected neighboring resources.

3.3 Prediction Tasks

Task 1: Current Anomaly Detection

$$\hat{a}_v^t = P(a_v^t = 1 \mid \mathcal{G}_{(t-L:t)})$$

Task 2: Multi-Horizon Failure Prediction

$$\hat{p}_{(v,\Delta)}^t = P(T_v \leq t + \Delta \mid T_v > t, \mathcal{G}_{(t-L:t)})$$

Possible forecasting horizons are 5 minutes, 30 minutes, 1 hour, 6 hours, and 24 hours.

Task 3: Failure-Rate Estimation

$$\hat{\lambda}_{(v,\Delta)}^t = -\ln(1 - \hat{p}_{(v,\Delta)}^t) / \Delta$$

This enables comparison of failure risk between heterogeneous components.

Task 4: Root-Cause and Propagation Analysis

$$\mathcal{S}_v = \{u \in \mathcal{V} : \text{Influence}(u \rightarrow v) > \eta\}$$

where η is an explanation threshold.

4. Proposed KubeKG-Forecast Architecture

4.1 Architectural Overview

The architecture consists of eight logical layers:

16. Kubernetes platform layer;
17. observability and telemetry layer;
18. dynamic knowledge-graph construction layer;
19. weak-supervision layer;
20. heterogeneous graph neural network layer;
21. temporal failure-forecasting layer;
22. explanation and root-cause layer;
23. proactive resilience-action layer.

The overall processing sequence is:

Kubernetes Resources → Prometheus/Thanos + Cilium/Hubble + Kubernetes Events → Dynamic Temporal Knowledge Graph in Neo4j → Unsupervised Weak Labeling → Heterogeneous Relational Graph Attention Network → Temporal Encoder → Current Anomaly + Future Failure Probability + Failure Rate → Root Cause + Propagation Path + Explainable Alert

4.2 Telemetry Collection

4.2.1 Prometheus and Thanos

Prometheus periodically scrapes resource and application metrics. Thanos provides long-term metric retention and cross-cluster querying. Each observation is associated with a timestamp:

$$m_v^t = (v, k, \text{value}, t)$$

where k is the metric name. Collected information is attached to the corresponding knowledge-graph node.

4.2.2 Cilium and Hubble

Cilium provides Kubernetes-native networking and visibility using eBPF. Hubble supplies network-flow observability. A communication observation can be represented as:

$$f_{ij}^t = (v_i, v_j, \text{protocol}, \text{srcPort}, \text{dstPort}, \text{bytes}, \text{packets}, \text{latency}, \text{status})$$

Unlike the kNN-generated graph used in GATAKU, KubeKG-Forecast directly converts communication into semantic edges:

$$(v_i, \text{communicatesWith}, v_j)$$

This preserves real operational dependencies.

4.3 Dynamic Knowledge Graph Construction

Neo4j is used as the operational graph store. Each resource is stored as a node with type-specific properties. A representative Pod record is shown below.

```
(:Pod {uid: "pod-17", namespace: "analytics", cpu_usage: 0.84, memory_usage: 0.72,
  restart_count: 4, anomaly_score: 0.81, failure_probability_30m: 0.67})
```

Relationships may be expressed as RUNS_ON, MANAGES, OWNS, COMMUNICATES_WITH, and MOUNTS. Each observation cycle produces an updated graph snapshot. A temporal sequence is therefore:

$$\mathbb{G}_t = \{\mathcal{G}_{(t-L+1)}, \dots, \mathcal{G}_t\}$$

Historical snapshots may be stored separately or represented through time-indexed feature properties.

5. Weakly Supervised Anomaly Learning

5.1 Motivation

Reliable labels are difficult to obtain in real Kubernetes environments. Operators may know that an outage occurred but may not possess exact labels for every Pod, Container, metric, and preceding time interval. The method therefore combines unsupervised pseudo-labels, explicitly injected fault labels, Kubernetes event labels, and verified operator incident labels.

5.2 Resource-Type-Specific Unsupervised Detection

Following the central principle of Paper 6, different Kubernetes resource types should not be compared indiscriminately. A Pod is evaluated relative to other Pods:

$$D_{\text{Pod}} = \{x_v^t : \tau(v) = \text{Pod}\}$$

Similarly, D_{Node} , $D_{\text{Container}}$, and D_{Service} are processed independently. An Isolation Forest produces an anomaly score:

$$s_v^t = \text{IF}(x_v^t)$$

Instead of forcing every observation into a hard class, two confidence thresholds are introduced:

$$\tilde{y}_v^t = 1 \text{ if } s_v^t \geq \theta_{\text{high}}; 0 \text{ if } s_v^t \leq \theta_{\text{low}}; \text{undefined otherwise}$$

Ambiguous observations are excluded from the initial supervised loss. This reduces error propagation from noisy pseudo-labels.

5.3 Incorporating Verified Labels

When an actual incident or controlled fault injection is known, its verified label overrides the pseudo-label:

$$y_v^t = y_{\text{verified}} \text{ if a verified label is available; otherwise } \tilde{y}_v^t$$

This creates a progressively improving training set.

6. Heterogeneous Relational Graph Neural Network

6.1 Initial Embedding

Because different node types contain different feature spaces, each resource type has a dedicated feature encoder:

$$h_v^0 = \varphi_{\tau(v)}(x_v)$$

where $\varphi_{\tau(v)}$ may be a multilayer perceptron.

6.2 Relation-Aware Message Passing

For relation r , messages from neighbor u to component v are:

$$m_{(u \rightarrow v)}(r, l) = W_r^l h_u^l$$

A relation-specific attention coefficient is computed as:

$$e_{uv}(r, l) = \text{LeakyReLU}(a_r^T [W_r h_u^l \parallel W_r h_v^l \parallel e_{uv}])$$

The normalized coefficient is:

$$\alpha_{uv}(r, l) = \exp(e_{uv}(r, l)) / \sum_{(k \in N_r(v))} \exp(e_{kv}(r, l))$$

Node representations are updated through:

$$h_v^{l+1} = \sigma(W_0 h_v^l + \sum_{(r \in \mathcal{R})} \sum_{(u \in N_r(v))} \alpha_{uv}(r, l) W_r^l h_u^l)$$

This allows the model to learn that different relationships have different operational meanings. `RUNS_ON` may capture infrastructure dependency, `COMMUNICATES_WITH` may capture network dependency, `MANAGES` may capture orchestration hierarchy, and `MOUNTS` may capture storage dependency.

6.3 Why the Native Graph Is Preferable to a kNN Graph

In a feature-similarity graph, two observations are connected because their numerical characteristics are similar. In the proposed approach, two components are connected because an actual Kubernetes or application relationship exists.

Suppose Pod A communicates with Pod B but their CPU and memory usage patterns differ. A kNN graph may separate them. However, failure of Pod A may directly affect Pod B through the service dependency. The semantic knowledge graph preserves this critical edge.

The proposed model can optionally include similarity edges as an additional relation, such as (Pod_i, similarTo, Pod_j), but similarity does not replace native topology.

7. Temporal Failure-Rate Prediction

7.1 Temporal Representation

A single graph snapshot cannot fully represent degradation. For component v , the graph encoder produces a sequence of embeddings over the historical window:

$$h_v^{(t-L+1)}, h_v^{(t-L+2)}, \dots, h_v^t$$

These embeddings are processed by a temporal encoder. Using a GRU:

$$z_v^t = \text{GRU}(h_v^{(t-L+1:t)})$$

An LSTM or temporal Transformer can be substituted.

7.2 Current Anomaly Head

$$\hat{a}_v^t = \sigma(W_a z_v^t + b_a)$$

7.3 Multi-Horizon Failure Head

$$\hat{p}_{(v, \Delta_j)}^t = \sigma(W_{(\Delta_j)} z_v^t + b_{(\Delta_j)})$$

Thus, the output may contain probabilities for 5 minutes, 30 minutes, 1 hour, 6 hours, and 24 hours.

Component	Current anomaly	Failure in 5 min	Failure in 30 min	Failure in 1 h
Node-03	0.42	0.18	0.51	0.76
Pod-17	0.91	0.64	0.88	0.95
Service-A	0.33	0.10	0.29	0.54

7.4 Discrete Failure Hazard

$$q_{(v,k)} = P(T_v = k \mid T_v \geq k, \mathbb{G}_t)$$

The cumulative probability of failure by interval K is:

$$P(T_v \leq K) = 1 - \prod_{k=1}^K (1 - q_{(v,k)})$$

This formulation is particularly useful when failure events are censored because many components may not fail during the observation period.

7.5 Component Failure Rate

$$\hat{\lambda}_{(v, \Delta)} = -\ln(1 - \hat{p}_{(v, \Delta)}) / \Delta$$

A platform-wide weighted failure rate can be computed as:

$$\Lambda_{\Delta}^{\text{platform}} = [\sum_{(v \in \mathcal{V})} w_v \hat{\lambda}_{(v, \Delta)}] / [\sum_{(v \in \mathcal{V})} w_v]$$

where w_v represents component criticality. A control-plane Node may receive greater weight than a temporary noncritical Pod.

8. Failure Propagation and Root-Cause Localization

8.1 Propagation Risk

For connected components u and v , propagation risk is defined as:

$$R_{(u \rightarrow v)}^t = \hat{a}_u^t \cdot \alpha_{uv} \cdot C_{uv}$$

where $\hat{a}_u^t$ is the anomaly probability of source component u , α_{uv} is learned graph attention, and C_{uv} is relation-specific criticality.

For a path $\pi = (v_1, v_2, \dots, v_n)$, the path propagation score may be:

$$R(\pi) = \prod_{i=1}^{n-1} R_{(v_i \rightarrow v_{i+1})}$$

The highest-scoring paths provide candidate explanations for cascading failures.

8.2 Hierarchical Kubernetes Reasoning

Suppose Pod P_7 exhibits severe memory growth. The knowledge graph may contain P_7 → ReplicaSet_3 → Deployment_A → Namespace_X and P_7 → Node_2. The system can report that Node-2 has a predicted 78% probability of failure within 30 minutes, with Pod-7 as the strongest root-cause candidate because of abnormal memory allocation and its learned influence on Node-2. This extends the root-cause-localization ability demonstrated in Paper 6 with GNN-derived influence scores.

9. Model Training

9.1 Multi-Task Objective

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{anomaly}} + \lambda_f \mathcal{L}_{\text{failure}} + \lambda_h \mathcal{L}_{\text{hazard}} + \lambda_c \mathcal{L}_{\text{consistency}} + \lambda_s \mathcal{L}_{\text{smooth}}$$

Current anomaly classification uses binary cross-entropy:

$$\mathcal{L}_{\text{anomaly}} = -\sum_v [y_v \log \hat{a}_v + (1-y_v) \log(1-\hat{a}_v)]$$

Because failures are rare, the failure head may use focal loss:

$$\mathcal{L}_{\text{failure}} = -\alpha(1-\hat{p})^\gamma y \log(\hat{p}) - (1-\alpha)\hat{p}^\gamma(1-y)\log(1-\hat{p})$$

A graph-smoothness regularizer may be defined as:

$$\mathcal{L}_{\text{smooth}} = \sum_{(u,v) \in \mathcal{E}} w_{uv} \|h_u - h_v\|_2^2$$

for relationships where operational similarity is expected.

9.2 Handling Class Imbalance

The following techniques are recommended:

- focal loss;
- class weighting;
- balanced incident sampling;
- temporal negative downsampling;
- threshold optimization on validation data;
- AUPRC-based model selection.

A random observation-level split should be avoided because neighboring time samples from the same incident can cause data leakage.

10. Online Monitoring and Model Updating

KubeKG-Forecast contains two main asynchronous workflows.

10.1 Update Graph Workflow

24. Query Kubernetes resource state.
25. Collect Prometheus metrics.
26. Collect Cilium/Hubble flow data.
27. Update nodes and relationships in Neo4j.
28. Generate node feature matrices.
29. Execute the deep graph model.
30. Estimate anomaly and failure probabilities.
31. Write predictions back to Neo4j.
32. Generate alerts when thresholds are exceeded.

10.2 Update Model Workflow

33. Retrieve historical graph observations.
34. Identify high-confidence anomalies using unsupervised detection.
35. Add verified incident labels.
36. Detect possible data drift.

37. Retrain or fine-tune the GNN and temporal model.
38. Recalibrate probability outputs.
39. Validate against a held-out temporal window.
40. Promote the model only if acceptance criteria are met.

This design retains the adaptive retraining principle introduced in Paper 6 while extending the classifier to a deep temporal graph model.

11. Algorithm

Algorithm 1: KubeKG-Forecast Online Failure Prediction

Input: Kubernetes cluster K, monitoring interval I, historical window L, forecasting horizons Δ . Output: anomaly probabilities, future failure probabilities, failure rates, and root causes.

1. Initialize Neo4j knowledge graph G.
2. Initialize telemetry collectors.
3. Load trained heterogeneous temporal GNN model M.
4. Every monitoring interval I:
 - 4.1 Query Kubernetes resource objects.
 - 4.2 Query Prometheus/Thanos metrics.
 - 4.3 Query Cilium/Hubble network flows.
 - 4.4 Query Kubernetes events.
 - 4.5 Update resource nodes in G.
 - 4.6 Update native Kubernetes edges in G.
 - 4.7 Update communication edges in G.
 - 4.8 Attach latest features to nodes and edges.
 - 4.9 Retrieve graph sequence $G[t-L+1:t]$.
 - 4.10 Encode type-specific input features.
 - 4.11 Perform heterogeneous relation-aware message passing.
 - 4.12 Apply temporal encoder to recent node embeddings.
 - 4.13 Predict current anomaly probability, multi-horizon failure probability, discrete failure hazard, and failure rate.
 - 4.14 For components above alert threshold, extract influential subgraph, rank root-cause candidates, identify likely propagation paths, and generate explanation.
 - 4.15 Write predictions and explanations back to Neo4j.
5. Periodically detect distribution drift, generate high-confidence pseudo-labels, combine pseudo-labels with verified incidents, and retrain and recalibrate the model.

12. Experimental Methodology

12.1 Proposed Testbed

A suitable experimental environment consists of one Kubernetes control-plane node, four or more worker nodes, k3s or standard Kubernetes, Traefik or NGINX ingress, Cilium networking, Hubble network observability, Prometheus, Thanos, Neo4j, Grafana, and PyTorch with PyTorch Geometric. The application workload should include multiple isolated Namespaces and replicated microservices to create sufficiently complex topology.

Example services include a front-end service, back-end API service, PostgreSQL database, data-processing service, cache service, and external-service connector.

12.2 Failure and Anomaly Scenarios

To evaluate operational reliability rather than only cybersecurity, the following scenarios should be injected:

Category	Scenarios
Resource failures	CPU saturation; memory leak; memory exhaustion; disk saturation; excessive disk I/O latency; Node memory pressure; Node disk pressure.
Container and Pod failures	OOMKilled; CrashLoopBackOff; repeated readiness-probe failures; repeated liveness-probe failures; application process crash.
Network failures	Packet loss; increased latency; bandwidth throttling; broken service connectivity.
Cybersecurity anomalies	DoS attack; port scan; ZAP vulnerability scan.

Including these scenarios extends the cybersecurity focus of GATAKU while retaining its valuable network-flow features.

12.3 Failure Definition

A clear distinction should be maintained between an anomaly and a failure. An anomaly is an observation or state that significantly deviates from learned normal behavior. A failure is an operational event resulting in one or more of component unavailability, unsuccessful health checks, container termination, Pod eviction, NodeNotReady state, service-level objective violation, severe request failure, or workload disruption.

$$\text{Anomaly} \neq \text{Failure}$$

The model must learn when an anomaly represents temporary benign variation and when it indicates an approaching failure.

12.4 Temporal Dataset Splitting

The recommended split is earliest 60% for training, next 20% for validation, and latest 20% for testing. The same injected incident should not appear across multiple partitions. A stronger evaluation uses leave-one-failure-type-out testing to assess generalization to previously unseen anomalies.

13. Baseline Models

The proposed model should be compared against the following baselines:

Model	Graph-aware	Temporal	Topology-aware	Failure prediction
Isolation Forest	No	No	No	Limited
One-Class SVM	No	No	No	Limited
DBSCAN	No	No	No	No
Decision Tree	No	No	Partial through features	Yes
Random Forest	No	No	Partial through features	Yes
SVM	No	No	No	Yes
GATAKU-style GCN	Yes	No	Artificial kNN graph	Primarily anomaly detection
Proposed KubeKG-Forecast	Yes	Yes	Native heterogeneous KG	Yes, multi-horizon

14. Evaluation Metrics

14.1 Classification Metrics

Precision, recall, F1 score, AUROC, and AUPRC should be reported. Because failures are rare, AUPRC is especially important.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

14.2 Probability Calibration

Failure probabilities must be calibrated. Recommended metrics are Brier score, Expected Calibration Error, and reliability diagrams. A model predicting a 70% failure probability should fail approximately 70% of the time across comparable predictions.

14.3 Early-Warning Lead Time

$$\text{LeadTime}_i = t_{\text{(failure,i)}} - t_{\text{(first-alert,i)}}$$

A practical prediction model should provide sufficient time for corrective action.

14.4 Root-Cause Metrics

Recommended measures include Top-1 root-cause accuracy, Top-3 root-cause recall, mean reciprocal rank, and path accuracy.

14.5 Operational Performance

Measure graph-update latency, inference latency, training time, memory usage, CPU usage, throughput, and maximum supported graph size.

15. Ablation Studies

Ablation A: Remove the Knowledge Graph

Use only flat component features. Purpose: determine the contribution of relational topology.

Ablation B: Replace Native Graph with kNN Graph

Purpose: directly compare the semantic knowledge graph with GATAKU-style similarity construction.

Ablation C: Remove Network Features

Purpose: determine the contribution of Cilium/Hubble flows.

Ablation D: Remove Temporal Encoder

Use only the current graph snapshot. Purpose: measure the value of degradation history.

Ablation E: Remove Weak Supervision

Train only with verified labels. Purpose: evaluate pseudo-labeling when labeled incidents are scarce.

Ablation F: Replace Heterogeneous GAT with Standard GCN

Purpose: quantify the benefit of resource- and relation-type awareness.

16. Feasibility Evidence from the Two Source Solutions

The numerical evidence below comes from the two source studies and must not be interpreted as experimental results of KubeKG-Forecast.

Source solution	Main model	Reported evidence	Relevance to new framework
GATAKU	Three-layer GCN	Mean single-class GCN F1 approximately 0.999 and multiclass F1 approximately 0.999 in the controlled dataset.	Demonstrates strong feasibility of GNNs for Kubernetes network anomaly classification.
GATAKU	GNExplainer	Identified influential nodes, edges, and network-flow features.	Supports explainable graph-based diagnosis.
Kubernetes resilience framework	Isolation Forest	Best reported silhouette value: 0.7941.	Supports unsupervised discovery of abnormal Kubernetes resource observations.
Kubernetes resilience framework	Decision Tree	Best reported F1: 0.8864.	Supports prediction based on labels generated from unsupervised anomaly discovery.
Kubernetes resilience framework	Neo4j dynamic graph	CPU- and memory-stress experiments localized problematic Nodes and Pods and traced related resources.	Supports topology-aware root-cause localization.

These complementary findings motivate the new architecture. The first study demonstrates that GNN-based learning is effective for Kubernetes network anomalies, but its graph is based primarily on feature similarity. The second demonstrates that Kubernetes itself can be modeled as a meaningful dynamic heterogeneous graph, but its learning models do not exploit deep graph message passing.

Native Kubernetes Knowledge Graph + Network-Flow Graph + Weak Supervision + Heterogeneous GNN + Temporal Deep Learning + Failure-Rate Forecasting

17. Research Hypotheses

H1: A native dynamic heterogeneous knowledge graph will outperform a flat feature representation for failure prediction.

H2: A native semantic graph will provide better generalization and root-cause localization than an artificial kNN similarity graph.

H3: Integrating Prometheus resource metrics with Cilium/Hubble network flows will outperform either modality individually.

H4: Temporal graph modeling will provide longer early-warning times than current-snapshot anomaly detection.

H5: Weak supervision based on high-confidence Isolation Forest labels will improve performance when verified failure labels are scarce.

H6: Relation-aware heterogeneous graph attention will outperform a homogeneous standard GCN in complex Kubernetes environments.

H7: Graph explanations will identify defective Pods and related higher-level Kubernetes resources with greater accuracy than metric-only feature importance.

18. Discussion

The proposed framework represents a shift from isolated anomaly classification toward proactive reliability intelligence.

GATAKU showed that Kubernetes network records could be converted into a graph and processed using GCNs. However, graph construction based on kNN similarity may connect observations because they are numerically close rather than because an actual operational dependency exists.

The dynamic graph architecture of Paper 6 provides the missing semantic structure. A Pod is not simply a vector of CPU and memory values. It exists within a hierarchy of Nodes, Namespaces, Deployments, ReplicaSets, Services, Containers, and storage resources.

KubeKG-Forecast preserves this structure. Furthermore, adding COMMUNICATES_WITH edges from Cilium/Hubble observations allows the graph to represent runtime dependencies that may not be visible from Kubernetes ownership objects alone.

The resulting graph therefore contains two complementary classes of relationships: control and structural relationships, such as MANAGES, OWNS, and RUNS_ON; and runtime communication relationships, such as COMMUNICATES_WITH. This combination is especially valuable for failure propagation.

For example, suppose a memory leak develops in a back-end Pod. The Pod may overload its Node while simultaneously increasing latency for several downstream services. A flat model may detect a high-memory outlier. A topology-aware GNN can reason that Pod memory degradation may lead to Node resource pressure, service latency, and dependent-service errors. A temporal model can further determine whether these changes are temporary or represent a progressive degradation trajectory.

The resulting output is substantially more useful than a binary anomaly label. Instead of reporting only that Node-3 is anomalous, the proposed system can produce a calibrated probability of operational failure within a future horizon, identify the strongest root-cause candidate, and show the most likely propagation path. Such explanations are more actionable for platform engineers.

19. Practical Deployment Considerations

19.1 Scalability

Large Kubernetes platforms may contain millions of graph observations. Scalability can be improved through neighborhood sampling, mini-batch graph learning, GraphSAGE-style sampling, time-window compression, hierarchical graph partitioning, component-specific subgraphs, and incremental embeddings.

19.2 Concept Drift

Normal behavior changes as applications are updated, workloads grow, hardware changes, services are added, and auto-scaling modifies topology. Consequently, the model should continuously monitor data drift and periodically update pseudo-labels and learned parameters.

19.3 Alert Fatigue

Anomaly-detection systems can produce excessive alerts. The proposed framework should combine calibrated failure probability, resource criticality, persistence duration, propagation risk, and confidence threshold. An alert priority score can be defined as:

$$\text{Priority}_v = \hat{p}(v, \Delta) \cdot \text{Criticality}_v \cdot \text{PropagationRisk}_v$$

19.4 Automated Resilience Actions

Future deployments may connect predictions to Kubernetes automation. Possible actions include rescheduling a Pod, increasing replicas, migrating workload, cordoning a Node, draining a Node, restarting a failed service, allocating additional memory, or triggering human investigation. However, fully automated remediation should require carefully defined confidence and safety controls.

20. Limitations

41. The unified KubeKG-Forecast model requires experimental validation. The reported GCN, Isolation Forest, and Decision Tree results belong to the two foundational studies and cannot be presented as results of the new unified framework.
42. Unsupervised pseudo-labels may contain noise. Confidence thresholds, expert verification, and robustness techniques are therefore required.
43. Kubernetes installations differ substantially in topology, workload, observability, networking, and application architecture. Generalization across clusters must be explicitly tested.
44. Graph neural network training can become computationally expensive for very large clusters.
45. Correlation does not necessarily imply causality. Attention weights and GNN explanations identify influential relationships but should not automatically be interpreted as definitive causal mechanisms.
46. Failure definitions must be standardized carefully. Temporary anomalies, application-level defects, security attacks, infrastructure failures, and service-level-objective violations are not equivalent events.
47. The architecture currently focuses primarily on Kubernetes. Extending the ontology to Hadoop, Spark, Kafka, HDFS, YARN, and other big-data frameworks requires additional component and relation types.

21. Extension to Hadoop and Other Big-Data Platforms

Although the proposed architecture is designed around Kubernetes, the same framework can represent Hadoop resources. Possible node types include NameNode, DataNode, ResourceManager, NodeManager, ApplicationMaster, Container, HDFSBlock, Job, and Queue.

(DataNode_7, stores, HDFSBlock_123)

(NodeManager_3, runsOn, Server_5)

(Job_8, scheduledBy, ResourceManager_1)

Features may include HDFS capacity, missing blocks, corrupt blocks, DataNode heartbeat status, NameNode RPC latency, JVM garbage-collection time, YARN container failures, disk utilization, and job retry count. A unified big-data platform knowledge graph could therefore simultaneously include Kubernetes and Hadoop components.

This supports the broader research objective of failure-rate prediction of large-scale big-data platforms using knowledge graphs and deep neural networks.

22. Conclusion

This paper proposed KubeKG-Forecast, a new dynamic knowledge graph and deep graph neural network framework for anomaly detection, root-cause localization, propagation analysis, and failure-rate prediction in Kubernetes-based big-data platforms.

The framework integrates two complementary research directions. From GATAKU, it adopts deep graph neural network processing, detailed inter-container network observability, engineered network features, and graph-based explainability. From the dynamic Kubernetes resilience approach, it adopts heterogeneous resource modeling, Neo4j-based topology storage, Prometheus monitoring, two-stage unsupervised and supervised learning, hierarchical resource analysis, and topology-aware localization.

The proposed method advances beyond both by constructing a semantic temporal knowledge graph containing native Kubernetes relationships and runtime communication edges, applying weakly supervised heterogeneous graph attention, learning degradation trajectories through temporal deep neural networks, and explicitly estimating future failure probability and failure rate over multiple horizons.

Kubernetes Telemetry + Network Flows + Dynamic Knowledge Graph + Weak Supervision + Heterogeneous GNN + Temporal Forecasting → Future Failure Rate

The proposed architecture offers a promising foundation for moving cloud-platform management from reactive anomaly response toward proactive, explainable, and topology-aware failure prevention.

Data Availability Statement

The new KubeKG-Forecast framework described in this manuscript is a proposed research architecture. No new unified experimental dataset has yet been generated as part of this manuscript draft. Future implementation should release

anonymized graph snapshots, telemetry features, fault-injection definitions, train-validation-test partitions, and model code where security and privacy constraints permit.

Conflict of Interest

The authors declare no conflict of interest.

References

- [1] M. Osswald, T. Schönenberger, G. Cantali, W. Soussi, and G. Gür, "Anomaly Detection in Microservices Architecture Using Graph Neural Networks," in *Proceedings of the 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, 2025, pp. 560-567, doi: 10.1109/PDP66500.2025.00085.
- [2] V. Anemogiannis, B. Andreou, K. Myrtollari, K. Panagidi, and S. Hadjiefthymiades, "Enhancing Kubernetes Resilience through Anomaly Detection and Prediction," *arXiv:2503.14114*, 2025.
- [3] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017.
- [4] P. Veličković et al., "Graph Attention Networks," in *International Conference on Learning Representations*, 2018.
- [5] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 8th IEEE International Conference on Data Mining*, 2008, pp. 413-422.
- [7] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, 1996.
- [8] J. Kosinska and M. Tobiasz, "Detection of Cluster Anomalies with ML Techniques," *IEEE Access*, vol. 10, pp. 110742-110753, 2022.
- [9] C.-W. Tien, T.-Y. Huang, C.-W. Tien, T.-C. Huang, and S.-Y. Kuo, "Kubanomaly: Anomaly Detection for the Docker Orchestration Platform with Neural Network Approaches," *Engineering Reports*, vol. 1, no. 5, e12080, 2019.
- [10] M. A. Rodriguez and P. Neubauer, "The Graph Traversal Pattern," *arXiv:1004.1001*, 2010.
- [11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [12] S. Naseer et al., "Enhanced Network Anomaly Detection Based on Deep Neural Networks," *IEEE Access*, vol. 6, pp. 48231-48246, 2018.
- [13] Q. Du, T. Xie, and Y. He, "Anomaly Detection and Diagnosis for Container-Based Microservices with Performance Monitoring," *EasyChair Preprint* 468, 2018.
- [14] B. Schölkopf, R. C. Williamson, A. Smola, J. Shawe-Taylor, and J. Platt, "Support Vector Method for Novelty Detection," in *Advances in Neural Information Processing Systems*, vol. 12, 1999.
- [15] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.